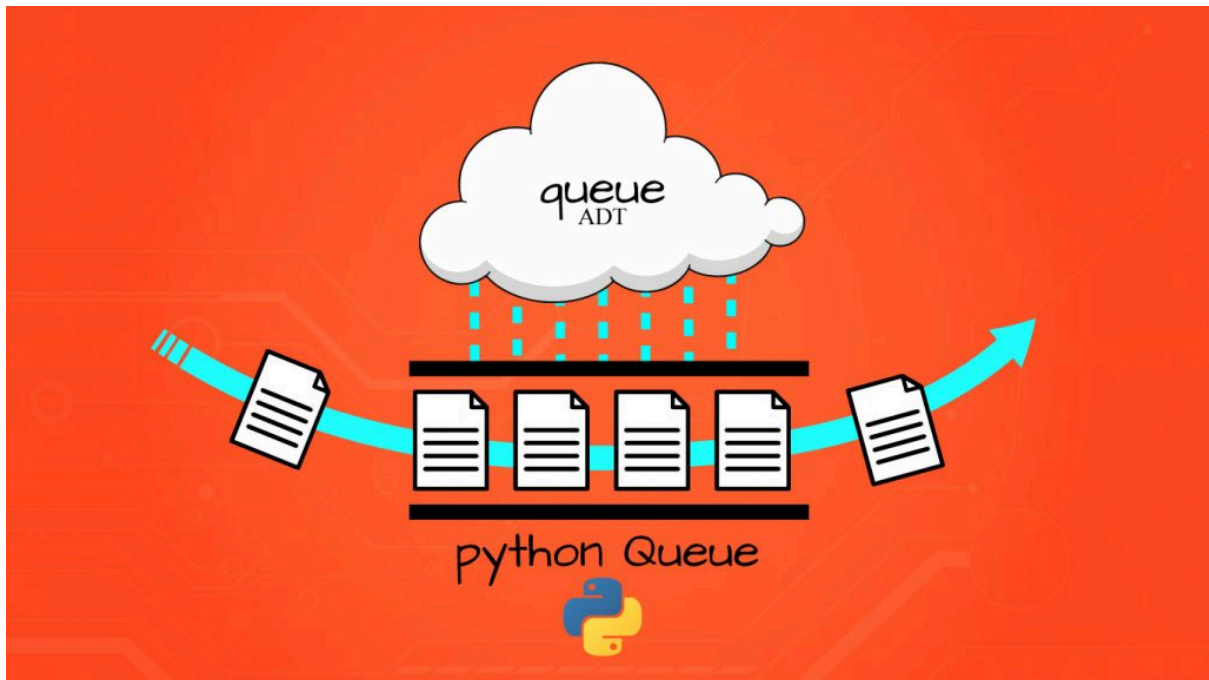


Tipo Abstracto de Datos (ADT)



Los tipos de datos abstractos son conceptos básicos en el mundo informático actual que ayudan a los desarrolladores a gestionar y organizar datos de forma eficiente. Ya sea que trabaje con listas, pilas o colas, comprender los tipos de datos abstractos es importante para aprender y resolver problemas relacionados con estructuras de datos y algoritmos. En este artículo, analizaremos qué es un tipo de datos abstracto, sus características, su importancia, sus desventajas, ejemplos como los TAD de lista, pila y cola , y sus aplicaciones .

¿Qué es un tipo de datos abstracto en las estructuras de datos?

Un tipo de datos abstracto en estructuras de datos es un concepto teórico que define una estructura de datos únicamente en términos de su comportamiento desde la perspectiva del usuario, en lugar de cómo

implementarla. En términos más simples, podemos decir que un TAD define qué operaciones se realizarán en los datos, pero no cómo se realizarán. Por lo tanto, el usuario de un TAD solo necesita estar familiarizado con las operaciones, no con la lógica de implementación.

Se denomina "abstracto" porque representa una vista de la estructura de datos independiente de la implementación. Además, en ADT, todos sus detalles internos están ocultos al usuario, lo que permite flexibilidad y modularidad en la programación. Ejemplos de tipos de datos abstractos en estructuras de datos son una lista, una pila, una cola, etc.

Características de un tipo de datos abstracto

Los tipos de datos abstractos son un ejemplo de un principio general en ingeniería de software, que recibe diversos nombres con significados ligeramente diferentes. Estos son algunos de los nombres que se utilizan para esta idea:

Abstracción: Los TAD proporcionan una visión general de la estructura de datos. Por lo tanto, los usuarios no necesitan saber cómo implementar las operaciones en las estructuras de datos; solo se proporciona información importante. - Omitir u ocultar detalles de bajo nivel con una idea más simple y de nivel superior.

Encapsulación: los detalles de implementación interna están ocultos y solo la interfaz está expuesta al usuario.- Construir un muro alrededor de un módulo para que el módulo sea responsable de su propio

comportamiento interno y los errores en otras partes del sistema no puedan dañar su integridad.

Operaciones bien definidas: Los tipos de datos abstractos especifican un conjunto de operaciones, como insertar, eliminar y buscar, que pueden realizarse con los datos. Estas operaciones están claramente definidas en términos de salida, entrada y comportamiento.

Independencia de implementación: diferentes estructuras de datos pueden implementar el mismo ADT.

Modularidad: Los ADT promueven el diseño modular, lo que ayuda a que los diferentes componentes de un programa se desarrollen y prueben de forma independiente - Dividir un sistema en componentes o módulos, cada uno de los cuales puede diseñarse, implementarse, probarse, razonarse y reutilizarse por separado del resto del sistema.

Reutilización: un tipo de datos abstracto se puede reutilizar en varios programas sin reescribir la implementación.

Ocultación de información: los usuarios no pueden acceder ni modificar los datos internos directamente, por lo que esto reduce el riesgo de errores accidentales y refuerza la integridad de los datos. - Ocultar detalles de la implementación de un módulo al resto del sistema, de modo que esos detalles se puedan cambiar más tarde sin cambiar el resto del sistema.

Mantenibilidad mejorada: Dado que los detalles de implementación están separados, los cambios en la implementación no afectan el código que utiliza el ADT. Esto facilita la depuración, las pruebas y la actualización del código.

Separación de preocupaciones: Hacer que una característica (o “preocupación”) sea responsabilidad de un solo módulo, en lugar de distribuirla entre varios módulos.

Como ingeniero de software, debería conocer estos términos, ya que los encontrará con frecuencia. El propósito fundamental de todas estas ideas es ayudar a lograr las tres propiedades importantes que nos importan en este curso: seguridad frente a errores, facilidad de comprensión y preparación para el cambio.

De hecho, ya Ud. ha encontrado algunas de estas ideas en cursos anteriores, en el contexto de escribir funciones que toman entradas y producen salidas:

Abstracción: una especificación es una abstracción en el sentido de que el cliente solo tiene que comprender sus precondiciones y poscondiciones para usarla, no el comportamiento interno completo de la implementación.

Modularidad: las pruebas unitarias y las especificaciones ayudan a convertir las funciones en módulos.

Encapsulación: Las variables locales de una función están encapsuladas, ya que solo la función puede usarlas o modificarlas. A diferencia de las variables globales, que son todo lo contrario, o las variables locales que apuntan a objetos mutables con alias, que también amenazan la encapsulación.

Ocultación de información: una especificación utiliza la ocultación de información para dejarle al implementador cierta libertad en cómo se implementa la función.

Separación de preocupaciones: una buena especificación de función es coherente , lo que significa que es responsable de una sola preocupación.

A partir de la clase de hoy, iremos más allá de las abstracciones para funciones y analizaremos también las abstracciones para datos . Sin embargo, veremos que las funciones seguirán desempeñando un papel crucial en la descripción de la abstracción de datos.

¿Por qué utilizar ADT?

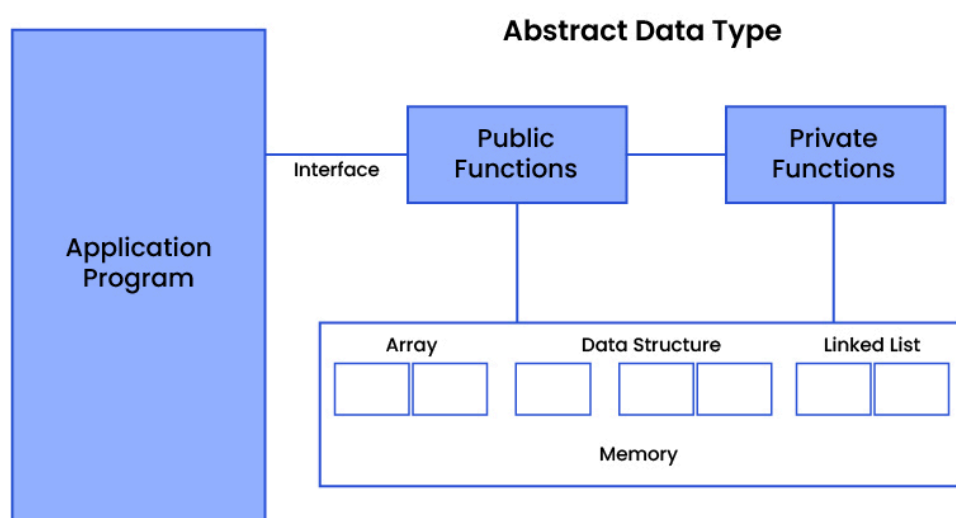
A continuación se presentan algunas razones por las que debería utilizar tipos de datos abstractos:

- Los tipos de datos abstractos **simplifican los sistemas complejos** al proporcionar una interfaz clara y ocultar los detalles de implementación.
- **Mejora la reutilización del código**, ya que los ADT se pueden utilizar en diferentes programas sin realizar cambios.

- **ADT mejora la capacidad de mantenimiento** ya que los cambios en la implementación interna no afectan el código externo.
- **Promueve la abstracción**, lo que permite a los desarrolladores centrarse en qué operaciones son necesarias en lugar de cómo se realizan.
- Los tipos de datos abstractos **admiten la programación modular**, lo que facilita el desarrollo, la prueba y la depuración de componentes individuales.
- También **garantiza la integridad de los datos** al restringir el acceso directo a la representación interna de los datos.

Modelo de tipo de datos abstracto

El modelo de tipos de datos abstractos representa un marco lógico que explica cómo se estructuran y manipulan los datos, sin proporcionar detalles de implementación. Se centra básicamente en las operaciones disponibles y su comportamiento, en lugar de en cómo se implementan en memoria.



Componentes del modelo ADT

Analicemos en detalle los componentes del modelo de tipo de datos abstracto:

1. Datos

Los datos son los valores o elementos que el tipo de dato abstracto está diseñado para almacenar y manipular. Describen el tipo y la estructura de la información almacenada, como los enteros en una pila o los pares clave-valor en un mapa.

Ejemplo: números enteros en una pila, caracteres en una cola.

2. Operaciones

Las operaciones son las funciones o métodos que definen cómo se puede acceder, insertar o manipular los datos dentro del TAD. Especifican qué acciones se pueden realizar, como insertar(), eliminar(), insertar() o extraer(), sin revelar cómo se implementan internamente.

Ejemplos: insertar(), eliminar(), buscar(), poner en cola(), empujar(), sacar().

3. Reglas o comportamiento

Las reglas o el comportamiento de un TAD nos indican cómo el tipo de dato abstracto interpreta sus operaciones y las restricciones lógicas que sigue. Por ejemplo, sabemos que una pila funciona según la regla LIFO

(Último en entrar, primero en salir), y una cola sigue la regla o principio FIFO (Primero en entrar, primero en salir). Estas reglas garantizan que los resultados del TAD funcionen y proporcionen los resultados esperados y predecibles.

4. Interfaz

La interfaz es el conjunto de operaciones públicas que permite a los usuarios interactuar con el TAD. Describe las acciones que se pueden realizar con los datos, sin definir la implementación subyacente. Garantiza que el usuario haya seguido correctamente el uso del tipo de dato abstracto sin necesidad de conocer su funcionamiento interno.

5. Implementación

La implementación consiste básicamente en el código subyacente y las estructuras de datos, como matrices, listas enlazadas, árboles, etc., que se utilizan para implementar el tipo de dato abstracto. Es una capa oculta entre el usuario y la implementación, lo que garantiza la abstracción y permite al usuario trabajar con el TAD sin conocer la implementación interna. Esto ofrece ventajas en términos de flexibilidad y facilidad de mantenimiento.

Clasificación de tipos y operaciones

Los tipos, ya sean integrados o definidos por el usuario, se pueden clasificar como **mutables** o **inmutables**. Los objetos de un tipo mutable pueden modificarse: es decir, proporcionan operaciones que, al ejecutarse, provocan resultados diferentes en otras operaciones sobre

el mismo objeto. También `Date` es mutable, porque se puede llamar `setMonth` observar el cambio con la `getMonth` operación. Pero `String` es inmutable, porque sus operaciones crean nuevos `String` objetos en lugar de modificar los existentes. A veces, un tipo se proporciona en dos formas: mutable e inmutable. `ReadonlyArray`, por ejemplo, es una forma inmutable de `Array`.

Las operaciones de un tipo abstracto se clasifican de la siguiente manera:

- **Creadores:** Crear nuevos objetos del tipo. Un creador puede tomar valores de otros tipos como argumentos, pero no un objeto del tipo que se está construyendo.
 - Una operación de creación a menudo se implementa como un *constructor*, como `new Date()`.
 - Pero un creador puede ser simplemente un método estático, como `Array.of()`. Un creador implementado como un método estático o una función independiente a menudo se denomina **fábrica**.
- **Productores:** También crea nuevos objetos del tipo, pero requiere uno o más objetos existentes del tipo como entrada.
 - `concat: String × String → String` es un productor: toma dos cadenas y produce una nueva cadena que representa su concatenación.
- **Observadores:** Toma objetos de tipo abstracto y devuelve objetos de un tipo diferente. Algunos observadores no aceptan otros argumentos, como:
 - `size: Map<K, V> → Number`

- ...y otros toman argumentos de otros tipos, además del valor del tipo abstracto:
 - `get: Map<K,V> × K → V`
- **Mutadores:** cambiar objetos. Por ejemplo,
 - `push: Array<T> × T → número`, por ejemplo, muta una matriz agregando un elemento al final (y de paso devuelve la nueva longitud de la lista).
 - Los mutadores suelen indicarse mediante un tipo de retorno `void`or` undefined``. Un método que devuelve ``void`` *debe* llamarse para algún tipo de efecto secundario, ya que de lo contrario no devuelve nada. Sin embargo, no todos los mutadores devuelven ``void``. Por ejemplo, ``devuelve el conjunto mismo``, de modo que se puedan `encadenar` `Set.add()` varias llamadas `.add()`

Ejemplos de tipos de datos abstractos

A continuación se muestran algunos ejemplos de tipos de datos abstractos, junto con algunas de sus operaciones, agrupadas por tipo.

`number` es el tipo numérico primitivo de TypeScript. `number` es inmutable, por lo que no tiene mutadores.

- creadores: los literales numéricos `0`, `1`, `2`, ...
- productores: operadores aritméticos `+`, `-`, `*`, `/`, `**`

- observadores: operadores de comparación `===`, `!==`, `<`, `>`
- mutadores: ninguno (es inmutable)

`Array` es el tipo de matriz de TypeScript. `Array` es mutable.

- creadores: `Array` constructor, sintaxis literal, `[ item1, item2, etc. ]` `Array.of` `Array.from`
- productores: `Array.concat`
- observadores: `length`, `[...]` sintaxis de indexación
- mutadores: `push`, `pop`, `shift`, `unshift`, `reverse`, `sort`

`string` es el tipo de cadena de TypeScript. `String` es inmutable.

- creadores: `String` constructor, sintaxis literal, método estático `"quoted text"` `String.fromCharCode`
- productores: `concat`, `substring`, `toUpperCase`
- observadores: `length`
- mutadores: ninguno (es inmutable)

Esta clasificación proporciona terminología útil, pero no es perfecta. Por ejemplo, en tipos de datos complejos, puede haber una operación que sea a la vez productor y mutador. Nos referiremos a dicho método como *productor* y mutador, pero algunos prefieren llamarlo simplemente mutador, reservando el término productor sólo para operaciones que no realizan mutación.

Un tipo abstracto se define por sus operaciones.

La idea esencial aquí es que un tipo de **dato abstracto se define por sus operaciones**. El conjunto de operaciones para un tipo T , junto con sus especificaciones, define completamente lo que entendemos por T .

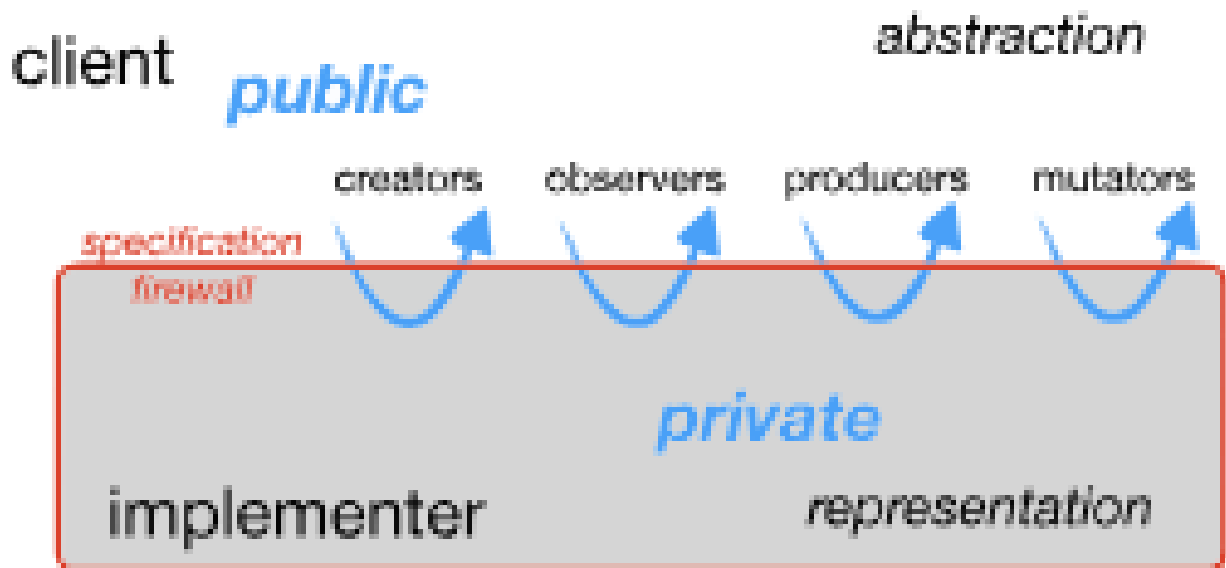
Entonces, por ejemplo, cuando hablamos del tipo `Array`, lo que queremos decir no es una lista enlazada, o un bloque de memoria, o una tabla hash, o cualquier otra estructura de datos específica que pueda representar una secuencia de valores.

En cambio, el tipo `Array` es un conjunto de valores opacos (los posibles objetos que pueden tener `Array` tipo) que satisfacen las especificaciones de todas las operaciones de `Array`: `[..]` (indexación), `length`, `push()`, etc. Los valores de un tipo abstracto son *opacos* en el sentido de que un cliente no puede examinar los datos almacenados en su interior, excepto según lo permitan las operaciones.

Ampliando nuestra metáfora de un **firewall de especificación**, usted podría imaginar los valores de un tipo abstracto como capas duras, que ocultan no sólo la implementación de una función individual, sino de un conjunto de funciones relacionadas (las operaciones del tipo) y los datos que comparten (los campos privados almacenados dentro de los valores del tipo).

Las operaciones del tipo constituyen su **abstracción**. Esta es la parte pública, visible para los clientes que utilizan el tipo.

Los campos de la clase que implementa el tipo, así como las clases relacionadas que ayudan a implementar una estructura de datos compleja, constituyen un conjunto particular de **representación de datos**. Esta parte es privada, visible sólo para el implementador del tipo.



an *abstract data type* is defined by its operations